Video script: GB Renewables Forecast Monitor (pilot)

Two recordings, one repo.

Video	Audience	Length	Goal
A. Customer overview	Trading desk / business sponsor	**≤ 5:00** (~620 words)	Impress: show the decision, the evidence, the honesty
B. Tech-lead walkthrough	Tech lead / reviewer	~18 min	Make review easy, transfer knowledge

Speaking pace assumed: ~130 words/min. `[SCREEN]` = what to show. `[SAY]` = voice-over. Numbers come from the README and notebook (last 12 months, 5% share).

Before recording: open the live app on the **Decision** tab at the latest complete day, keep `notebooks/analysis.ipynb` and the repo open in a second window, and run `.venv/bin/python -m pytest -q` once so the green run is cached (19 tests, <1 s).

Video A: Customer overview (5:00)

0:00–0:30 · The problem

`[SCREEN]` Title slide, then the Decision tab.

`[SAY]` "Every morning, before 11:00 UK time, your desk has to decide how much power to buy for tomorrow in the day-ahead auction. Wind is the biggest unknown. If wind under-delivers, you buy the gap later at the imbalance price.

We built a pilot that answers three questions: How much do we buy? How good are the forecasts we rely on? And what does being wrong actually cost us?"

0:30–1:45 · The decision screen

`[SCREEN]` Decision tab. Point at the three lines, then the totals, then the flags.

`[SAY]` "This is the Decision screen. For any delivery day, it gives you megawatts to buy for every half-hour, at three levels: expected, a recommended policy level, and a conservative level.

The key design rule: it only uses information that existed before the auction gate. Each number is traceable to the forecast publish time, and the screen shows those times. No hindsight leaks in.

Below that, two early-warning flags: one for a large forecast revision overnight, one for a steep wind ramp. These are computed against their own history, so a flag means 'unusual for this system', not a fixed threshold.

For past days, the screen shows what you actually needed. So you can check the tool against reality, day by day."

1:45–2:45 · Cost and risk

`[SCREEN]` Risk & cost tab: frontier chart, then cumulative excess £.

`[SAY]` "This tab turns forecast error into pounds. We replayed twelve months and compared each buying strategy against perfect hindsight.

Two findings. First, the model's 90th-percentile quantity covered the real need on 95% of days. Second, and this is the honest part: over a year, the average saving from buying a bigger buffer is small, around one percent. Prices barely moved with forecast error. The value of a buffer is protection against the bad tail days, and the screen shows that as a cost-versus-risk curve, so you choose your risk appetite with a number, not a feeling.

The recommendation is only labelled 'recommended' when it has beaten the simple rules on its own out-of-sample record. A rolling version we tried first did not, so we replaced it."

2:45–3:45 · Forecast accuracy

`[SCREEN]` Wind accuracy tab: forecast vs actual with band, then MAE vs lead time. Switch to Solar briefly.

`[SAY]` "The accuracy tabs answer 'can we trust these forecasts?'. Wind, with a forecast band built only from errors known at the time. Error against lead time. Daily bias. And the same for solar.

One result worth a headline: the apparent 1.4 gigawatt wind over-forecast is not a weather problem. It's a scope difference. The forecast covers all wind, including embedded wind. The demand it offsets covers only transmission wind. Once you compare like with like, bias is about zero. That alone prevents a systematic over-buy of wind credit.

We also found that the DGWS day-ahead forecast is published about six hours after the gate. It can be measured, but it can't be traded on. The tool says so on screen."

3:45–4:30 · Built to run itself

`[SCREEN]` GitHub Actions run, then the Vercel deployment, then the notebook header.

`[SAY]` "It runs itself. A scheduled job refreshes the public Elexon data before the gate each morning, commits it, and the site redeploys. No API keys, no database, no manual steps. Access is behind login.

Every number on screen is reproducible. A notebook recomputes the headline results using the app's own code."

4:30–5:00 · Close

`[SCREEN]` Back to Decision tab, then a one-line "Next" slide.

`[SAY]` "What you've seen is a working pilot on public data. The next step is to plug in your own demand and portfolio, so 'MW to buy' becomes your number, not a national proxy, and to price against the real auction instead of the market index. We'd like thirty minutes to agree that scope. Thank you."

> **Presenter notes.** Do not claim the buffer "saves money on average": the data says otherwise and a sharp customer will check. Do say the tool quantifies the tail. If asked "why 5%?": it is a proxy for your share of national demand (`PORTFOLIO_SHARE`), a one-line config change.

Video B: Tech-lead walkthrough (~18:00)

Open with the agenda on screen. Cues are timestamps for the edit, not hard limits.

0:00–1:30 · Scope and constraints

`[SCREEN]` README top, then the "Layout" tree.

`[SAY]` "Quick framing. This is a two-phase pilot. Phase 1 was a wind forecast monitor: accuracy against actuals, gate-aware. Phase 2 added MW-to-buy, solar and twelve months of history. Phase 3, just built, adds purchase-decision metrics: £ backtest, uncertainty models and a purchase policy.

Constraints that shaped everything: public Elexon data only, no API key, no database, one deployable, and a result a trader can audit.

Roughly 1,800 lines of Python and JSX excluding data, and 19 tests."

1:30–5:00 · Architecture

`[SCREEN]` Draw or show this diagram, then walk it left to right.

```
Elexon BMRS (public)
      │  scripts/fetch_data.py   (GitHub Action 09:45 + 00:30 UTC)
      ▼
monitor/data/<dataset>/<YYYY-MM>.json.gz     ← the "database": 3.2 MB, in git
      │  monitor/data.py   load + shape once at cold start
      ▼
monitor/core.py   (pure: time, selection, metrics, bands, MW-to-buy)
monitor/risk.py   (pure: residual models, £ backtest, policy, flags)
      │
api/index.py      FastAPI: /api/meta /decision /risk /accuracy
      │  vercel.json rewrite
      ▼
web/src           React + Vite + Recharts: Decision / Risk / Accuracy tabs
```

`[SAY]` "Five decisions to call out.

One: **the data snapshot lives in git**, as monthly gzip partitions. Total 3.2 MB. That gives us a free database, free history, reviewable diffs on data changes, and the app has no runtime dependency on Elexon being up. Trade-off: it's a snapshot, refreshed twice a day, not live. Fine for a day-ahead decision.

Two: **logic is pure and separated from the web layer.** `core.py` and `risk.py` take data in and return data out. No I/O, no framework. That's why the same code runs in the API, in tests, and in the notebook.

Three: **one Vercel function.** Vercel turns every Python file under `api/` into a function, so only `index.py` lives there, and the logic sits in `monitor/` and is bundled in with `includeFiles`. We hit that as a bug early; it's now a rule in the layout.

Four: **the frontend is thin.** It renders what the API returns. No forecasting logic in JS.

Five: **Docker is for local and self-hosted runs only.** Production is Vercel plus a GitHub Action. Two paths, one codebase."

5:00–8:30 · Analysis design: what makes the numbers trustworthy

`[SCREEN]` `monitor/core.py`: `gate_cutoff`, `select`, `rolling_band`. Then README "Method".

`[SAY]` "The analysis rests on four rules. If you review nothing else, review these.

Rule 1, time. Delivery days are Europe/London days, so clock-change days have 46 or 50 half-hours. The gate is strictly before 11:00 London time on D-1, BST-aware. See `gate_cutoff` and `day_bounds`. Tests cover both cutoffs and both clock-change days.

Rule 2, forecast selection. In gate mode: latest publish strictly before the gate. In horizon mode: latest publish at or before target minus H. `select` is the single entry point; every metric goes through it.

Rule 3, no leakage. The uncertainty band for day D uses only errors from D-31 to D-2: errors fully known at the gate. `rolling_band` and `days_back` enforce this, and there's a test that the band never sees D-1 or later. Every 'out-of-sample' number in the app depends on this.

Rule 4, comparing like with like. WINDFOR covers all wind; National Demand is net of embedded generation, so the right actual is transmission wind, FUELHH. Compare against that, and WINDFOR looks 1.45 GW high. Compare against the wider AGWS and bias is +0.04 GW. We correct the forecast against FUELHH before sizing the buy, and solar is not subtracted because NDF already nets it.

Sign convention: error is actual minus forecast. Negative means a shortfall, which means buy more."

8:30–12:00 · Phase 3 model: risk and purchase policy

`[SCREEN]` `monitor/risk.py`, top docstring, then `Risk.model_quantiles`, `excess_cost`, `cvar`.

`[SAY]` "Phase 3 asks: how much *should* we buy, in pounds.

Define the residual: forecast residual is NDF minus WINDFOR at the gate; actual residual is demand minus FUELHH wind; error is the difference. Purchase at quantile q is the forecast plus the q-th quantile of recent error, floored at zero.

Three competing uncertainty models: wind-only, demand-plus-wind over thirty days, and conditional, which buckets by wind tercile and evening peak and falls back to demand-plus-wind when a bucket has fewer than a hundred samples. That `MIN_N` is marked with a `ponytail:` comment because it's a deliberate simplification with a known ceiling.

A champion is chosen daily by trailing-90-day pinball loss. Out-of-sample, conditional wins: pinball 476 versus 544 for wind-only, with P90 coverage at 88.8%. In practice it's champion on every day.

Cost: excess £ versus perfect hindsight, per half-hour, is need minus buy, times imbalance price minus day-ahead price, times half an hour. Policy: per month, choose the quantile minimising mean daily excess plus lambda times CVaR95, with lambda from `RISK_AVERSION`.

I want to be direct about a negative result, because it affects how you read the screen. The first version used a rolling 90-day window and lost to both always-P50 and always-P90 on its own track record, so it's replaced with an expanding window, and the UI only says 'recommended' when the policy has at least 60 out-of-sample days and matches or beats both baselines. Currently it settles on P50. The honest takeaway is that prices barely react to forecast error, correlation 0.06, so the buffer buys tail protection, not average savings."

12:00–14:00 · API and frontend

`[SCREEN]` `api/index.py`, then `web/src/main.jsx` and `Decision.jsx`.

[SAY] "The API is four GET routes: `meta`, `decision`, `risk`, `accuracy`. Route handlers only parse and validate. Config comes from two env vars, `PORTFOLIO_SHARE` and `RISK_AVERSION`, read through `env_number`, which clamps to a valid range so a bad value fails safe. Accuracy ranges cap at 400 days to bound compute.

Data and the `Risk` object are built once at cold start and cached, so request handlers are cheap.

Frontend: Vite and React, Recharts for charts. `main.jsx` owns tabs; each view is one file; shared chart components are in `charts.jsx`. No state library, no router. A reviewer can read the whole frontend in about fifteen minutes."

14:00–15:30 · Tests and verification

[SCREEN] Terminal: run pytest. Then open `tests/test_core.py` and `test_risk.py` briefly.

[SAY] "Nineteen tests, under a second. They are targeted at the places a silent error would be expensive: BST and GMT gate cutoffs, the strict less-than at the gate, 46 and 50 period days, the band never using D-1, the MW-to-buy floor at zero, the solar night filter, and the risk maths.

Beyond unit tests, the notebook recomputes the headline numbers using the app's own modules, and I cross-checked them against the API output. If the notebook and the API ever disagree, that's a bug."

15:30–16:45 · Operations and deployment

[SCREEN] `refresh.yml`, `vercel.json`, Vercel dashboard (Deployment Protection).

[SAY] "The Action runs at 09:45 UTC, after the NDF and WINDFOR runs and before the gate in both GMT and BST, and at 00:30 to complete yesterday's actuals. It commits only if partitions changed, and the push triggers a Vercel redeploy. A `concurrency` group prevents overlapping runs.

Production is protected with Vercel Authentication on all deployments. Pushes to `main` go to production; branches get previews. Env vars are set per environment with `vercel env add`."

16:45–18:00 · Known limits, risks, and how to review

[SCREEN] README "Still out of scope" and the data-quality notes.

[SAY] "Know these before you review.

- **Price proxy.** Day-ahead price is the Elexon APX market index, not the EPEX or N2EX auction price, which isn't on Elexon. £ figures are indicative.
- **Share is a proxy.** The 5% portfolio share stands in for the client's real demand.
- **DGWS timing.** Its day-ahead run arrives after the gate; the UI flags it, and it must never feed the buy.
- **Source gaps.** AGWS has gaps in summer 2026, and there's a real DGWS solar outlier on 26 April, kept deliberately.
- **Snapshot, not live.** Refreshed twice daily.
- **Weak month.** The P10 band covers 88.4% against a 90% target; September 2026 fell to 74% as seasons turned.

Suggested review order, about 45 minutes: README 'Method', then `core.py` selection and bands, then `risk.py`, then the tests, then `api/index.py`, then the frontend. If you only have ten minutes: `gate_cutoff`, `rolling_band`, `Risk.model_quantiles`, and the leakage test.

Out of scope, by decision: client demand data, other confidence levels, demand-forecast accuracy, audit trail, a database. Thank you."

Recording checklist

- [ ] Tests pass on camera (`19 passed`).
- [ ] Live app opens without auth friction (log in beforehand).
- [ ] Notebook outputs are saved so cells don't need to run live.
- [ ] Zoom browser to 125%; hide bookmarks and notifications.
- [ ] Record A in one take, with a title and a closing slide. Record B in segments by timestamp so errors are cheap to redo.
- [ ] Consistency: say "P90 of need" (not "P90 forecast"), "gate" (not "cutoff"), "excess £ versus perfect hindsight" throughout.